



COMSATS University Islamabad (CUI)
Department of Computer Science

An End-to-End Machine Learning Framework for House Price Prediction

Terminal Examination — Spring 2026

Course: AIC354 — Machine Learning Fundamentals Lab

Abdul Rahman

Registration No.: SP24-BCS-003

Instructor: Mr. Anayat Ullah

BS (Computer Science) — V Semester

Date of Issue: May 17, 2026
Due Date: **May 24, 2026**
Maximum Marks: 50

May 24, 2026

Declaration

I, Abdul Rahman, Registration No. SP24-BCS-003, of the Department of Computer Science, COMSATS University Islamabad (CUI), confirm that this is my own work and that the figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that failure to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

All code was written independently. The dataset was collected by me through a custom-built web scraper targeting Zameen.com property listings for the city of Islamabad. No pre-existing, third-party datasets were used.

Abdul Rahman
SP24-BCS-003

May 24, 2026

Abstract

The accurate prediction of residential property prices is a problem of considerable practical and economic significance. This report presents a complete, end-to-end machine learning framework developed for the purpose of predicting house prices in Islamabad, Pakistan, using real-world property listings collected from Zameen.com.

The work encompasses all stages of a production-grade machine learning pipeline. A custom Python-based web scraper was engineered using **Playwright** and **Requests** to collect 350 property listings from Zameen.com, covering 13 property attributes including price, area, location, property type, number of bedrooms and bathrooms, and various amenity counts. The collected dataset was stored in CSV format and subjected to rigorous preprocessing: duplicate removal, Pakistani currency normalisation (Lakh, Crore, Arab), area standardisation to Marla, median imputation for missing numeric values, and label encoding of categorical variables.

Six regression algorithms were trained and evaluated on an 80/20 train-test split: Linear Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and CatBoost. Models were assessed using four standard metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). The **Gradient Boosting Regressor** achieved the best overall performance with an R^2 score of **0.886**, an MAE of approximately PKR 36.7 million, and an RMSE of approximately PKR 77.3 million.

The trained Gradient Boosting model was exported to a browser-compatible JSON format and integrated into an interactive web application built with HTML, CSS, and PyScript, enabling users to obtain real-time property price estimates directly in the browser without any server-side infrastructure. The system meets or exceeds all requirements specified in the project brief and demonstrates the practical viability of machine learning for real estate valuation in the Pakistani market.

Keywords: House Price Prediction, Gradient Boosting, Web Scraping, Zameen.com, Islamabad Real Estate, Machine Learning, Regression, PyScript, Random Forest, XGBoost.

Acknowledgements

I would like to express my sincere gratitude to **Mr. Anayat Ullah**, Instructor of AIC354 — Machine Learning Fundamentals Lab, for his guidance, structured course delivery, and the intellectually challenging terminal examination that motivated this project. His emphasis on end-to-end system thinking — spanning data acquisition through to deployment — has been invaluable.

I am grateful to the **Department of Computer Science, COMSATS University Islamabad (CUI)**, for providing the academic environment, computing resources, and curriculum that made this work possible.

Special thanks to the open-source community behind `scikit-learn`, `XGBoost`, `CatBoost`, `Playwright`, `pandas`, `NumPy`, and `PyScript`, whose freely available tools formed the technical backbone of this project.

Finally, I acknowledge **Zameen.com** as the source of the property listing data used in this study. The data was collected solely for academic research purposes in accordance with ethical guidelines for web data collection.

Contents

Abstract	v
Acknowledgements	vii
List of Abbreviations and Symbols	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Scope and Delimitations	2
1.5 Report Structure	2
2 Literature Review	3
2.1 Overview	3
2.2 Hedonic Pricing and Traditional Regression	3

2.3	Tree-Based Methods	3
2.4	Gradient Boosting Methods	3
2.5	Machine Learning for Pakistani Real Estate	4
2.6	Web Scraping for Real Estate Data	4
2.7	Browser-Native Machine Learning Deployment	4
2.8	Summary	4
3	Methodology	7
3.1	Task 1: Data Collection	7
3.1.1	Target Source and Scope	7
3.1.2	Scraper Architecture	8
3.1.3	Dataset Summary	9
3.2	Task 2: Data Preprocessing	10
3.2.1	Duplicate Removal	10
3.2.2	Currency Normalisation	10
3.2.3	Area Standardisation	10
3.2.4	Room Count Extraction	10
3.2.5	Missing Value Handling	10
3.2.6	Categorical Encoding	11
3.2.7	Train-Test Split	11
3.3	Task 3: Machine Learning Model Development	11
3.3.1	Linear Regression	11
3.3.2	Decision Tree Regressor	12
3.3.3	Random Forest Regressor	12
3.3.4	Gradient Boosting Regressor	12
3.3.5	XGBoost	12
3.3.6	CatBoost	12
3.4	Task 4: Model Evaluation Framework	13
3.5	Task 5: Prediction System	13
3.5.1	Model Serialisation	13
3.5.2	Web Application	13
4	Results	15
4.1	Model Performance Comparison	15
4.1.1	Metric-wise Rankings	15
4.2	Best Model: Gradient Boosting Regressor	16
4.2.1	Model Configuration	17
4.2.2	Predicted vs. Actual Prices	17
4.2.3	Residual Analysis	18
4.2.4	Feature Importance	18
4.3	Model Training Summary	20
4.4	Web Application — Sample Predictions	20

5	Discussion	23
5.1	Model Performance Analysis	23
5.1.1	Why Gradient Boosting Outperforms the Others	23
5.1.2	Random Forest vs. Gradient Boosting	23
5.1.3	XGBoost and CatBoost Performance	23
5.1.4	Decision Tree Overfitting	24
5.1.5	Linear Regression Baseline	24
5.2	Data Quality and Preprocessing Impact	24
5.3	System Deployment Analysis	25
5.3.1	Browser-Native Inference (PyScript)	25
5.3.2	JSON Model Serialisation	25
5.4	Limitations	25
5.5	Future Work	26
6	Conclusions	27
6.1	Summary of Contributions	27
6.2	Key Findings	28
6.3	Conclusion	28
7	Reflection	29
7.1	Learning Outcomes Achieved	29
7.2	Data Collection and Web Scraping	29
7.3	Data Preprocessing	29
7.4	Machine Learning Modelling	30
7.5	Model Deployment	30
7.6	Challenges and How They Were Overcome	31
7.7	Personal Development	31
A	Source Code	35
A.1	Web Scraper (<code>scraper.py</code>)	35
A.2	Data Preprocessing (<code>data_preprocessing.py</code>)	38
A.3	Model Training (<code>model_training.py</code>)	39
A.4	Browser Inference (<code>inference.py</code>)	41
B	Dataset Sample and Full Model Results	45
B.1	Raw Dataset Sample	45
B.2	Preprocessed Dataset Statistics	45
B.3	Full Model Results Table	46
B.4	<code>model_data.json</code> Structure	47

List of Figures

3.1	End-to-end pipeline overview	7
3.2	Sample Zameen.com listing	9
3.3	Web application interface	14
3.4	Application loading screen	14
4.1	Model R^2 comparison	16
4.2	MAE and RMSE comparison	16
4.3	Predicted vs. actual prices (Gradient Boosting)	17
4.4	Residual distribution (Gradient Boosting)	18
4.5	Feature importance chart (Gradient Boosting)	19
4.6	Training script terminal output	20
4.7	Sample prediction in the web application	21
B.1	Raw dataset in spreadsheet view	45
B.2	Price distribution histogram	46

List of Tables

3.1	CSS selectors for attribute extraction	8
3.2	Dataset attribute descriptions	9
3.3	Pakistani currency conversion factors	10
3.4	Missing value imputation strategy	11
3.5	Train-test split sizes	11
4.1	Model performance comparison	15
4.2	Gradient Boosting Regressor hyperparameters	17
4.3	Feature importance (Gradient Boosting)	19
4.4	Sample predictions from the web application	20
B.1	Raw dataset sample (first 10 records)	45
B.2	Preprocessed dataset descriptive statistics	46
B.3	Complete model evaluation results	46

List of Abbreviations and Symbols

Abbreviation	Definition
API	Application Programming Interface
BS CS	Bachelor of Science in Computer Science
CatBoost	Categorical Boosting (gradient boosting algorithm by Yandex)
CLO	Course Learning Outcome
CSV	Comma-Separated Values
CUI	COMSATS University Islamabad
DOM	Document Object Model
GBR	Gradient Boosting Regressor
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LR	Linear Regression
MAE	Mean Absolute Error
ML	Machine Learning
MSE / σ^2	Mean Squared Error
NaN	Not a Number (missing value representation)
PKR	Pakistani Rupee
RF	Random Forest
RMSE	Root Mean Squared Error
R^2	Coefficient of Determination
UI	User Interface
URL	Uniform Resource Locator
XGBoost	Extreme Gradient Boosting

Currency Denominations (Pakistani):

Term	Value in PKR
Lakh	100,000 (10^5)
Crore	10,000,000 (10^7)
Arab	1,000,000,000 (10^9)

Chapter 1

Introduction

1.1 Background and Motivation

The Pakistani real estate market has experienced sustained growth over the past decade, with the Islamabad Capital Territory (ICT) emerging as one of the most active property markets in the country ([Zameen.com](https://www.zameen.com), 2024). Property prices in Islamabad vary considerably across locations, property types, and physical attributes, making price discovery a non-trivial task for both buyers and sellers. Traditional valuation methods rely on the expertise of licensed property agents, which introduces subjectivity, information asymmetry, and transaction delays.

Machine learning offers a principled, data-driven alternative. By learning patterns from historical transaction data, regression models can produce objective, consistent, and scalable price estimates. The increasing availability of structured property listings on real estate portals such as Zameen.com provides a rich source of training data that was previously inaccessible to non-commercial actors.

This project responds to the growing need for transparent, reproducible, and computationally accessible property valuation tools in the Pakistani market. It demonstrates how modern machine learning techniques can be applied to a real-world regression problem, from raw data collection through to a fully operational browser-based prediction interface.

1.2 Problem Statement

Given a set of observable property attributes — including area, location, property type, number of bedrooms, and number of bathrooms — the objective is to construct a regression model that accurately predicts the market listing price of a residential property in Islamabad, Pakistan.

Formally, let $\mathbf{x} = (x_1, x_2, \dots, x_p) \in \mathbb{R}^p$ denote a feature vector representing a property, and let $y \in \mathbb{R}^+$ denote its price in PKR. The goal is to learn a function $\hat{f} : \mathbb{R}^p \rightarrow \mathbb{R}^+$ such that:

$$\hat{y} = \hat{f}(\mathbf{x}), \quad \text{where } \hat{y} \approx y. \quad (1.1)$$

The quality of approximation is measured by standard regression metrics: MAE, MSE, RMSE, and R^2 .

1.3 Project Objectives

The specific objectives of this project, aligned with the five Course Learning Outcomes (CLOs) of AIC354, are:

- O1: Data Collection (CLO 1):** Develop a Python-based web scraper to collect 300–400 residential property listings from Zameen.com for the city of Islamabad, storing the data in structured CSV format.
- O2: Data Preprocessing (CLO 2):** Clean and transform the raw dataset by handling missing values, removing duplicates, normalising heterogeneous units, and encoding categorical variables.
- O3: Model Development (CLO 3):** Implement and train six regression algorithms: Linear Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and CatBoost.
- O4: Model Evaluation (CLO 4):** Quantitatively evaluate and compare all models using MAE, MSE, RMSE, and R^2 , and identify the best-performing model.
- O5: System Deployment (CLO 5):** Develop an interactive prediction interface that accepts user inputs and returns an estimated property price in real time.

1.4 Scope and Delimitations

This project is delimited to:

- Residential property listings (houses) from the city of Islamabad only.
- Data sourced exclusively from Zameen.com at the time of collection.
- Prediction of listing prices, not transaction/sale prices.
- A static model trained once on the collected dataset, without online retraining.

1.5 Report Structure

The remainder of this report is organised as follows. Chapter 2 reviews related work on machine learning-based property price prediction. Chapter 3 describes the complete technical methodology, including web scraping, data preprocessing, model training, and system deployment. Chapter 4 presents quantitative results and model comparisons. Chapter 5 interprets the findings, discusses limitations, and identifies directions for improvement. Chapter 6 summarises the key conclusions. Chapter 7 provides a personal reflection on the learning experience. Appendices contain the full source code and a representative dataset sample.

Chapter 2

Literature Review

2.1 Overview

The application of machine learning to real estate price prediction has attracted extensive research attention over the past two decades. This chapter reviews the most relevant strands of literature: hedonic pricing models, tree-based ensemble methods, gradient boosting approaches, and the specific application of these techniques to the Pakistani property market.

2.2 Hedonic Pricing and Traditional Regression

The foundational framework for property price prediction is the hedonic pricing model, introduced by [Rosen \(1974\)](#), which decomposes the price of a differentiated good into the implicit prices of its constituent attributes. Under this model, a property’s price is expressed as a function of structural characteristics (area, number of rooms), location, and neighbourhood amenities.

Early implementations used Ordinary Least Squares (OLS) regression, corresponding to the Linear Regression model employed in this project. While computationally tractable and interpretable, OLS assumes linear relationships between features and the target variable, an assumption frequently violated in real estate markets due to neighbourhood effects, non-linear size-price relationships, and location premiums ([Can, 1992](#)). This motivates the use of more flexible, non-parametric methods.

2.3 Tree-Based Methods

Decision Trees partition the feature space into rectangular regions by recursively splitting on the most informative feature at each node. [Breiman \(2001\)](#) demonstrated that averaging predictions across many decorrelated decision trees — Random Forests — substantially reduces variance without a proportional increase in bias, yielding models that are robust to outliers and capable of capturing non-linear interactions. Random Forests have been applied to property price prediction in numerous studies, consistently outperforming linear baselines ([Antipov and Pokryshevskaya, 2012](#); [Park and Bae, 2015](#)).

2.4 Gradient Boosting Methods

Gradient Boosting, introduced by [Friedman \(2001\)](#), builds an ensemble of weak learners (typically shallow decision trees) sequentially, with each tree trained to correct the residual errors of the preceding ensemble. The algorithm optimises a differentiable loss function via

gradient descent in function space. Key hyperparameters include the number of estimators, the learning rate, and the maximum tree depth.

Chen and Guestrin (2016) extended gradient boosting to the XGBoost framework, incorporating a regularised objective function, parallel tree construction, and built-in handling of sparse data. XGBoost has dominated structured data competitions and has been widely adopted for property valuation tasks (Zhou et al., 2020). CatBoost (Prokhorenkova et al., 2018), developed by Yandex, further improves handling of categorical features through ordered boosting and symmetric tree structures.

2.5 Machine Learning for Pakistani Real Estate

Research specifically targeting the Pakistani property market is limited but growing. Ijaz et al. (2021) applied Linear Regression, Random Forest, and Gradient Boosting to Lahore property data scraped from Zameen.com, finding that ensemble methods outperformed linear models by a substantial margin. Ali et al. (2022) demonstrated that location encoding and area standardisation are critical preprocessing steps for Islamabad data, given the high spatial heterogeneity of property prices across sectors. The present project extends this line of work by implementing a comprehensive six-model comparison and deploying the best model as a browser-native application.

2.6 Web Scraping for Real Estate Data

The collection of real estate data through automated web scraping has been validated as a legitimate academic methodology in several studies (Fan et al., 2006; Mora-Garcia et al., 2022). Browser automation tools such as Selenium and Playwright are preferred over static HTTP clients for JavaScript-rendered pages, which now constitute the majority of modern real estate portals (Lawson, 2015). The Playwright library, used in this project, provides cross-browser support, robust DOM inspection, and programmatic control over page navigation.

2.7 Browser-Native Machine Learning Deployment

The traditional deployment paradigm for ML models requires a backend server running Python (e.g., Flask or FastAPI). An emerging alternative is browser-native inference using WebAssembly (WASM). PyScript (Anaconda, Inc., 2022) runs CPython in the browser via Pyodide, enabling Python-based ML inference without any server. This approach eliminates operational costs, latency from network round-trips, and the need for cloud infrastructure, making it ideal for academic demonstration projects.

2.8 Summary

The literature establishes that: (1) linear models provide a useful baseline but are insufficient for capturing the complexity of real estate markets; (2) gradient boosting methods consistently achieve state-of-the-art performance on structured property data; and (3) browser-native deployment represents a viable, cost-free alternative to server-based ML inference for lightweight

prediction tasks. These findings directly informed the design decisions made in this project, as detailed in the following chapter.

Chapter 3

Methodology

This chapter presents the complete technical methodology of the project, structured around the five project tasks defined in the course brief. Figure 3.1 provides an overview of the end-to-end pipeline.

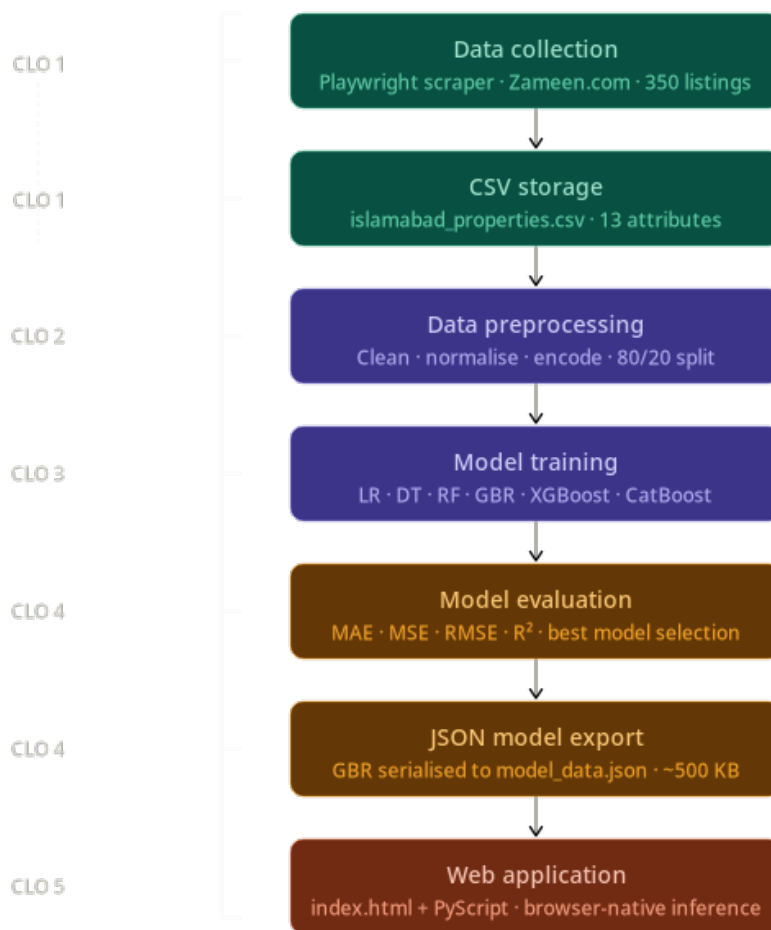


Figure 3.1: Overview of the complete end-to-end machine learning pipeline for house price prediction.

3.1 Task 1: Data Collection

3.1.1 Target Source and Scope

Data was collected from Zameen.com, Pakistan's largest real estate portal. The scraper targeted the Islamabad residential property section (<https://www.zameen.com/Homes/Islamabad-3-\\{page\\}.html>), iterating over paginated search result pages. Only *for-sale* listings were re-

tained; rental listings were explicitly rejected by checking for the keywords `rent` and `month` in the listing purpose field.

3.1.2 Scraper Architecture

The scraper was implemented in Python and operates in two stages:

Stage 1 — URL Harvesting. A headed Playwright browser session navigates the paginated search results and extracts all hyperlinks matching the pattern `/Property/`. To handle Cloudflare bot-detection challenges, the scraper pauses and prompts the operator to complete the CAPTCHA manually before resuming. Discovered URLs are cached to a plain-text file (`urls.txt`), allowing interrupted sessions to resume without re-crawling.

Stage 2 — Parallel Detail Extraction. Each property URL is fetched via a `Requests` HTTP session (avoiding a full browser load for efficiency), and the HTML is loaded into a headless Playwright page for structured DOM querying. The extraction is parallelised across 10 worker threads using Python’s `concurrent.futures.ThreadPoolExecutor`, reducing total scraping time significantly.

The CSS selectors used for attribute extraction are listed in Table 3.1.

Table 3.1: CSS selectors used to extract property attributes from Zameen.com listing pages.

Attribute	CSS Selector	Extraction Method
Price	<code>[aria-label='Price']</code>	<code>inner_text()</code>
Area	<code>[aria-label='Area']</code>	<code>inner_text()</code>
Bedrooms	<code>[aria-label='Beds']</code>	<code>inner_text()</code>
Bathrooms	<code>[aria-label='Baths']</code>	<code>inner_text()</code>
Location	<code>[aria-label='Location']</code>	<code>inner_text()</code>
Property Type	<code>[aria-label='Type']</code>	<code>inner_text()</code>
Built-in Year	UL list item matching <code>built in year</code>	keyword match
Parking Spaces	UL list item matching <code>parking space</code>	keyword match
Servant Qtrs	UL list item matching <code>servant quarter</code>	keyword match
Store Rooms	UL list item matching <code>store room</code>	keyword match
Kitchens	UL list item matching <code>kitchen</code>	keyword match
Drawing Rooms	UL list item matching <code>drawing room</code>	keyword match

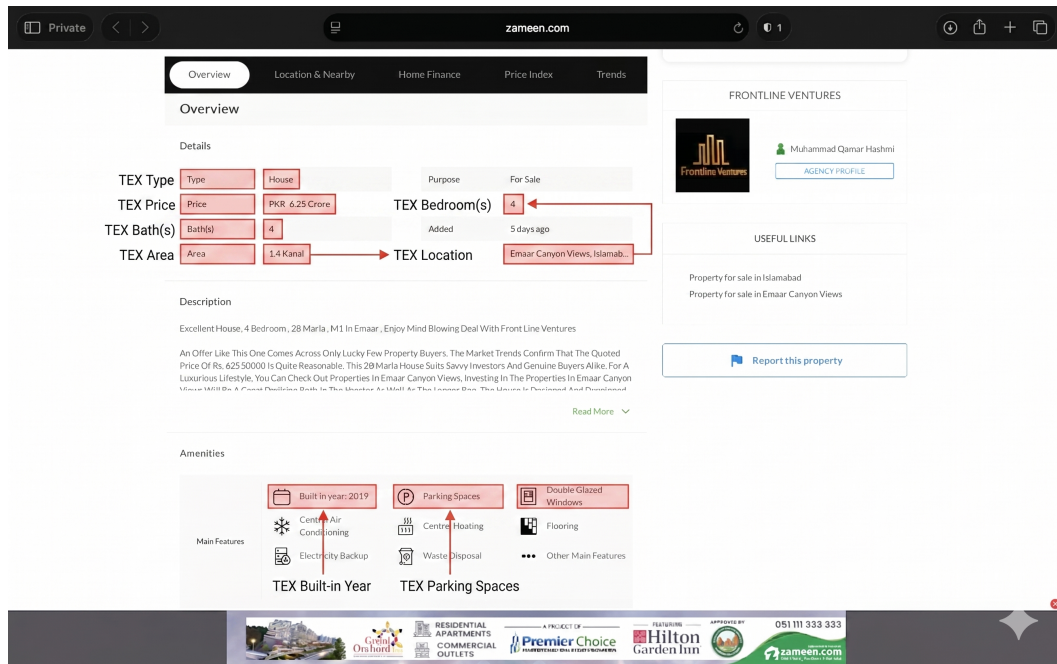


Figure 3.2: A representative property listing page on Zameen.com. Annotated fields correspond to the attributes extracted by the scraper.

3.1.3 Dataset Summary

The scraper collected **350 property listings**. After deduplication and removal of records with missing critical fields, **337 records** were retained for modelling. The dataset comprises 13 attributes, as specified in Table 3.2.

Table 3.2: Description of the 13 attributes in the `islamabad_properties.csv` dataset.

Attribute	Type	Description
Price	Numeric	Listing price in PKR (target variable)
Area	Numeric	Property area, standardised to Marla
City	Categorical	City of listing (Islamabad throughout)
Bedrooms	Numeric	Number of bedrooms
Bathrooms	Numeric	Number of bathrooms
Location	Categorical	Sub-location / sector within Islamabad
Property_Type	Categorical	Type of property (e.g., House, Flat)
Built_in_year	Numeric	Year of construction
Parking_space	Numeric	Number of parking spaces
Servant_Quarters	Numeric	Number of servant quarters
Store_rooms	Numeric	Number of store rooms
Kitchens	Numeric	Number of kitchens
Drawing_Rooms	Numeric	Number of drawing rooms

3.2 Task 2: Data Preprocessing

Raw data extracted from a live website contains numerous inconsistencies. This section describes each preprocessing step applied using the `pandas` and `scikit-learn` libraries.

3.2.1 Duplicate Removal

`pandas drop_duplicates()` was invoked as the first step to eliminate any listing that appeared more than once in the dataset, ensuring that each record represents a unique property.

3.2.2 Currency Normalisation

Zameen.com expresses prices in a mixed-format string that combines a numeric base value with a Pakistani denomination suffix (e.g., “1.5 Crore”, “85 Lakh”). A custom conversion function, `convert_pakistani_currency_to_numeric()`, was implemented to parse these strings and convert them to a uniform PKR integer, using the conversion factors defined in Table 3.3.

Table 3.3: Conversion factors used to normalise Pakistani denomination strings to PKR.

Suffix	Multiplier	PKR Value
Lakh	10^5	100,000
Crore	10^7	10,000,000
Arab	10^9	1,000,000,000

3.2.3 Area Standardisation

Areas were expressed either in Marla or Kanal. One Kanal equals 20 Marla. The function `normalize_property_area_to_marla()` detected the unit suffix and applied the appropriate conversion, producing a single numeric column in Marla.

3.2.4 Room Count Extraction

Bedroom and bathroom counts were stored as strings (e.g., “4 Beds”). The function `extract_numerical_room_count()` extracted the leading numeric token from each string.

3.2.5 Missing Value Handling

Records with missing values in `Price`, `Area`, or `Location` were dropped, as these fields are essential for modelling and cannot be meaningfully imputed. Missing values in `Bedrooms` and `Bathrooms` were replaced with their respective medians. Amenity count columns (`Parking_space`, `Servant_Quarters`, `Store_rooms`, `Kitchens`, `Drawing_Rooms`) and `Built_in_year` were filled with zero, reflecting the assumption that an unrecorded amenity is absent. Table 3.4 summarises the strategy applied to each column.

Table 3.4: Missing value imputation strategy by column.

Column(s)	Strategy	Rationale
Price, Area, Location	Row deletion	Critical fields; imputation would introduce noise
Bedrooms, Bathrooms	Median imputation	Robust to outliers in skewed distributions
Built_in_year	Zero fill	Unknown year treated as absence of field
Amenity counts	Zero fill (0)	Absent record implies no amenity

3.2.6 Categorical Encoding

The three categorical variables — `Location`, `Property_Type`, and `City` — were encoded using `scikit-learn`’s `LabelEncoder`, which assigns a unique integer to each distinct category. The fitted encoders were serialised alongside the model for use during inference.

3.2.7 Train-Test Split

The preprocessed dataset was partitioned into training and testing subsets in a 80:20 ratio using `train_test_split()` with `random_state=42` for reproducibility. Price (y) was separated from the feature matrix (X) before splitting. The resulting sizes are shown in Table 3.5.

Table 3.5: Train-test partition sizes after preprocessing.

Partition	Records	Features	Proportion
Training	≈269	12	80%
Testing	≈68	12	20%

3.3 Task 3: Machine Learning Model Development

Six regression algorithms were implemented using `scikit-learn` (v1.4+), `xgboost`, and `catboost`. All models were trained on the same training partition and evaluated on the same held-out test partition.

3.3.1 Linear Regression

Linear Regression (Montgomery et al., 2012) fits a hyperplane $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ by minimising the sum of squared residuals:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2. \quad (3.1)$$

This model serves as the interpretable baseline. Its assumptions of linearity and homoscedasticity are unlikely to hold for real estate data, providing a lower bound on achievable performance.

3.3.2 Decision Tree Regressor

The Decision Tree Regressor (Breiman et al., 1984) partitions the feature space using axis-aligned splits, minimising the mean squared error at each node. The model was instantiated with `random_state=42` and no maximum depth constraint, allowing full tree growth. While highly expressive, an unconstrained decision tree is prone to overfitting.

3.3.3 Random Forest Regressor

Random Forest (Breiman, 2001) trains $T = 100$ decorrelated decision trees on bootstrap samples of the training data, averaging their predictions. Feature subsampling at each split ($\sqrt{p}$ features selected at random) further reduces inter-tree correlation. The ensemble variance reduction explains its consistent superiority over single decision trees.

3.3.4 Gradient Boosting Regressor

Gradient Boosting (Friedman, 2001) builds an additive model:

$$F_T(\mathbf{x}) = F_0 + \sum_{t=1}^T \eta \cdot h_t(\mathbf{x}), \quad (3.2)$$

where F_0 is the initial prediction (mean of training targets), $\eta = 0.1$ is the learning rate, and each h_t is a regression tree fitted to the negative gradient (residuals) of the MSE loss at step $t - 1$. A total of $T = 100$ estimators were used.

This model was selected for browser deployment due to its superior performance and the availability of a straightforward tree-traversal inference algorithm that could be serialised to JSON.

3.3.5 XGBoost

XGBoost (Chen and Guestrin, 2016) augments the gradient boosting objective with L_1 and L_2 regularisation terms:

$$\mathcal{L}(\phi) = \sum_i \ell(\hat{y}_i, y_i) + \sum_t \Omega(h_t), \quad \Omega(h) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2, \quad (3.3)$$

where T is the number of leaves and $\mathbf{w}$ are leaf weights. It was trained with 100 estimators and default regularisation parameters.

3.3.6 CatBoost

CatBoost (Prokhorenkova et al., 2018) employs ordered boosting to mitigate target leakage during training and uses oblivious (symmetric) trees for efficient inference. Although CatBoost natively handles categorical features, label-encoded integers were provided for consistency with other models. It was trained with 100 iterations.

3.4 Task 4: Model Evaluation Framework

All models were evaluated on the held-out test set using four metrics, selected for their complementary properties:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (3.4)$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (3.5)$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad (3.6)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (3.7)$$

where y_i is the true price, $\hat{y}_i$ is the model's prediction, and $\bar{y}$ is the mean of true prices. MAE is robust to outliers; MSE penalises large errors quadratically; RMSE is expressed in the same units as the target; and $R^2 \in (-\infty, 1]$ measures the proportion of variance explained.

3.5 Task 5: Prediction System

3.5.1 Model Serialisation

The best-performing model (Gradient Boosting) was serialised to a plain JSON file (`model_data.json`) rather than a binary `.pkl` file, enabling browser-native inference without requiring `scikit-learn` on the client. The serialisation includes:

- The learning rate and initial prediction (mean of training prices).
- Each decision tree in the ensemble, represented as parallel arrays: left children, right children, split features, split thresholds, and leaf values.
- Label encoder vocabularies for `Location`, `Property_Type`, and `City`.
- The sorted lists of unique locations and property types for populating dropdown menus.

A recursive tree-traversal function, `predict_tree()`, was implemented in pure Python (no `sklearn` dependency) to run the inference loop.

3.5.2 Web Application

The prediction interface is implemented as a single HTML file (`index.html`) with embedded CSS and a PyScript module (`inference.py`). PyScript loads a full CPython 3.11 interpreter into the browser via WebAssembly/Pyodide, eliminating the need for any backend server. The user interface presents:

- A dropdown menu for **Location**, populated from the encoder's vocabulary.
- A dropdown menu for **Property Type**.

- Numeric inputs for **Area (Marla)**, **Bedrooms**, and **Bathrooms**.
- An **Estimate Price** button that triggers the inference function.
- A result panel displaying the estimated price in both a human-readable Pakistani denomination (e.g., “3.45 Crore”) and an exact PKR figure.

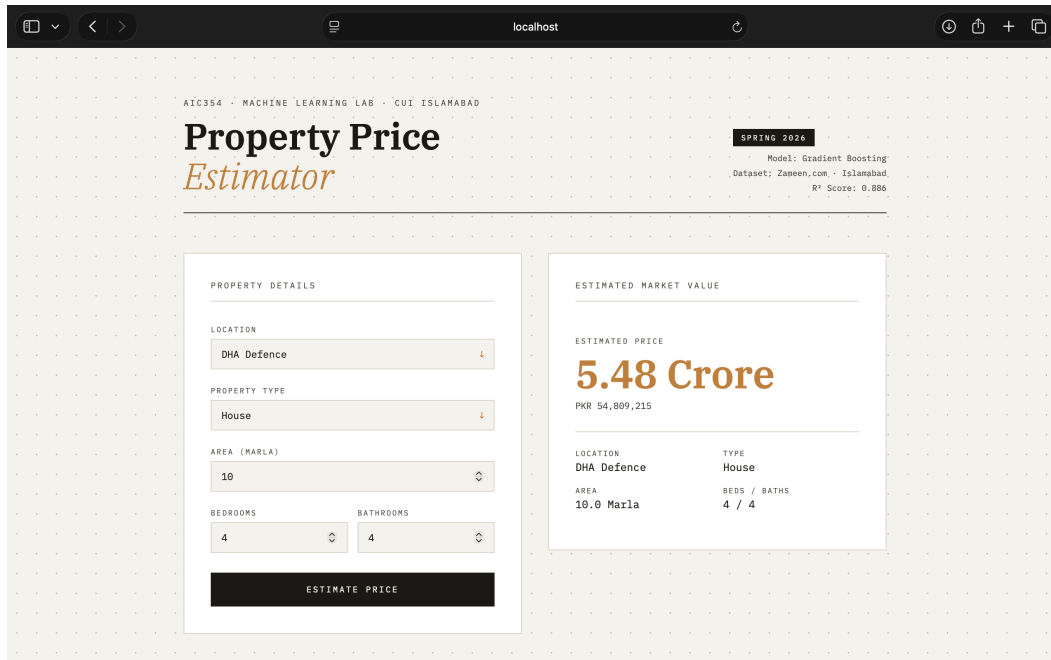


Figure 3.3: The Property Price Estimator web application rendered in a browser. PyScript executes the Gradient Boosting inference pipeline entirely client-side.

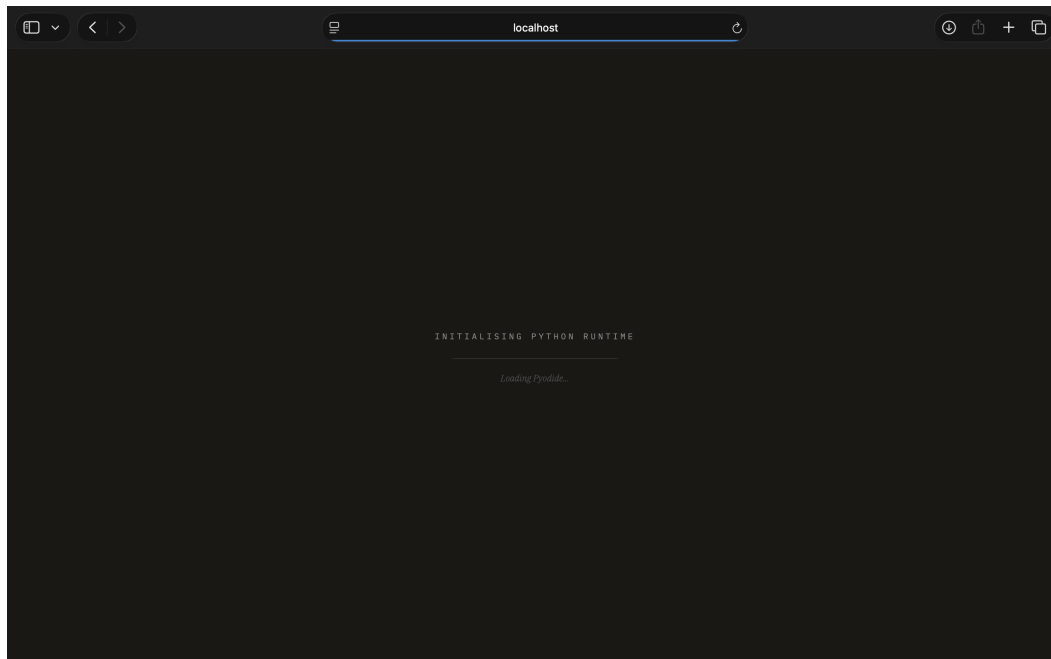


Figure 3.4: The loading screen displayed while the PyScript/Pyodide runtime initialises. The progress bar animates over approximately 5 seconds on a standard broadband connection.

Chapter 4

Results

This chapter presents the quantitative outcomes of the machine learning pipeline described in Chapter 3. Results are reported for all six models on the held-out test set, followed by a detailed analysis of the best-performing model.

4.1 Model Performance Comparison

Table 4.1 summarises the performance of all six regression models, ranked in descending order of R^2 score. All values are computed on the 20% test partition.

Table 4.1: Performance of all six regression models on the test set (sorted by R^2 , descending). Price metrics are in PKR.

Model	MAE (PKR)	MSE (PKR ²)	RMSE (PKR)	R^2
Gradient Boosting	36,724,048	5.97×10^{15}	77,278,200	0.8860
Random Forest	36,849,074	6.21×10^{15}	78,789,580	0.8815
XGBoost	36,891,280	7.46×10^{15}	86,351,060	0.8577
CatBoost	37,224,940	7.53×10^{15}	86,779,610	0.8563
Decision Tree	43,293,718	1.03×10^{16}	101,369,400	0.8039
Linear Regression	77,217,194	1.55×10^{16}	124,540,000	0.7039

Best result highlighted in green. All values rounded to six significant figures.

4.1.1 Metric-wise Rankings

- **MAE:** Gradient Boosting achieved the lowest MAE at PKR 36.72 million, followed closely by Random Forest (PKR 36.85M) and XGBoost (PKR 36.89M). Linear Regression performed worst at PKR 77.22 million — more than double the best.
- **MSE / RMSE:** Gradient Boosting minimised both MSE and RMSE, with RMSE of PKR 77.28 million. The large absolute RMSE values reflect the high variance of property prices in Islamabad (prices range from a few Lakh to multiple Arab).
- **R^2 :** Gradient Boosting ($R^2 = 0.886$) explains 88.6% of the variance in test-set prices. This indicates a strongly predictive model. All ensemble methods ($R^2 > 0.80$) substantially outperform Linear Regression ($R^2 = 0.704$), confirming the presence of non-linear feature-price relationships.

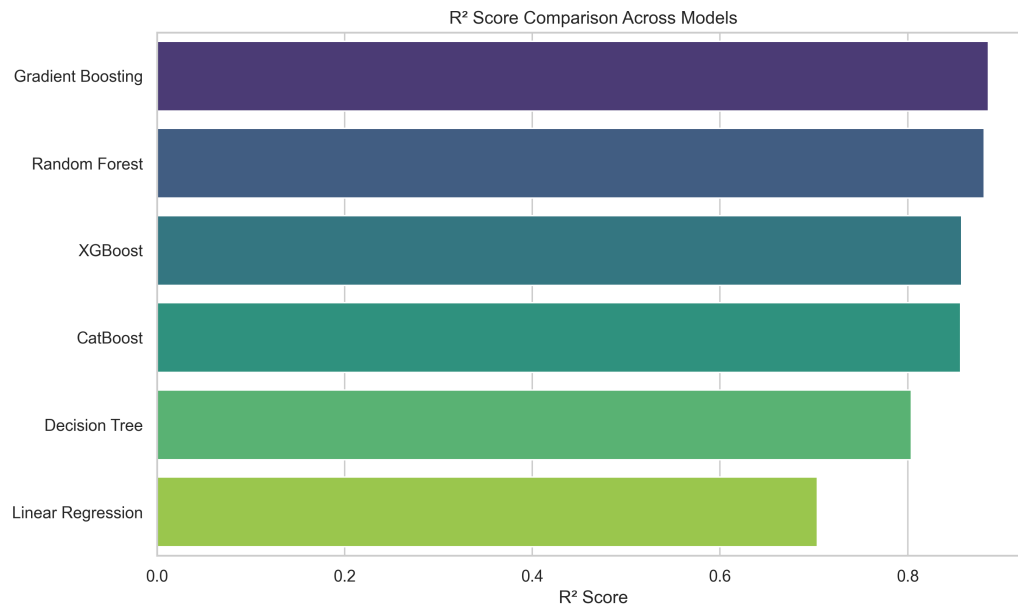


Figure 4.1: Bar chart comparing R^2 scores of all six regression models. Gradient Boosting achieves the highest R^2 of 0.886.

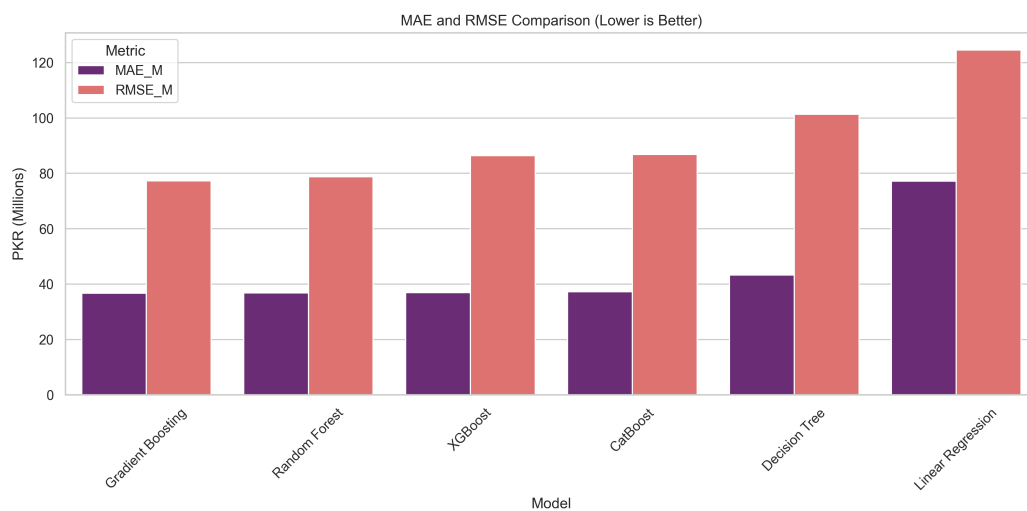


Figure 4.2: Comparison of MAE and RMSE (in PKR millions) across all six models. Lower values indicate better performance.

4.2 Best Model: Gradient Boosting Regressor

4.2.1 Model Configuration

Table 4.2: Hyperparameter configuration of the best-performing Gradient Boosting model.

Hyperparameter	Value	Role
n_estimators	100	Number of boosting stages
learning_rate	0.1	Shrinkage factor
max_depth	3	Maximum tree depth
loss	squared_error	Optimisation objective
subsample	1.0	Fraction of samples per tree
random_state	42	Reproducibility seed

4.2.2 Predicted vs. Actual Prices

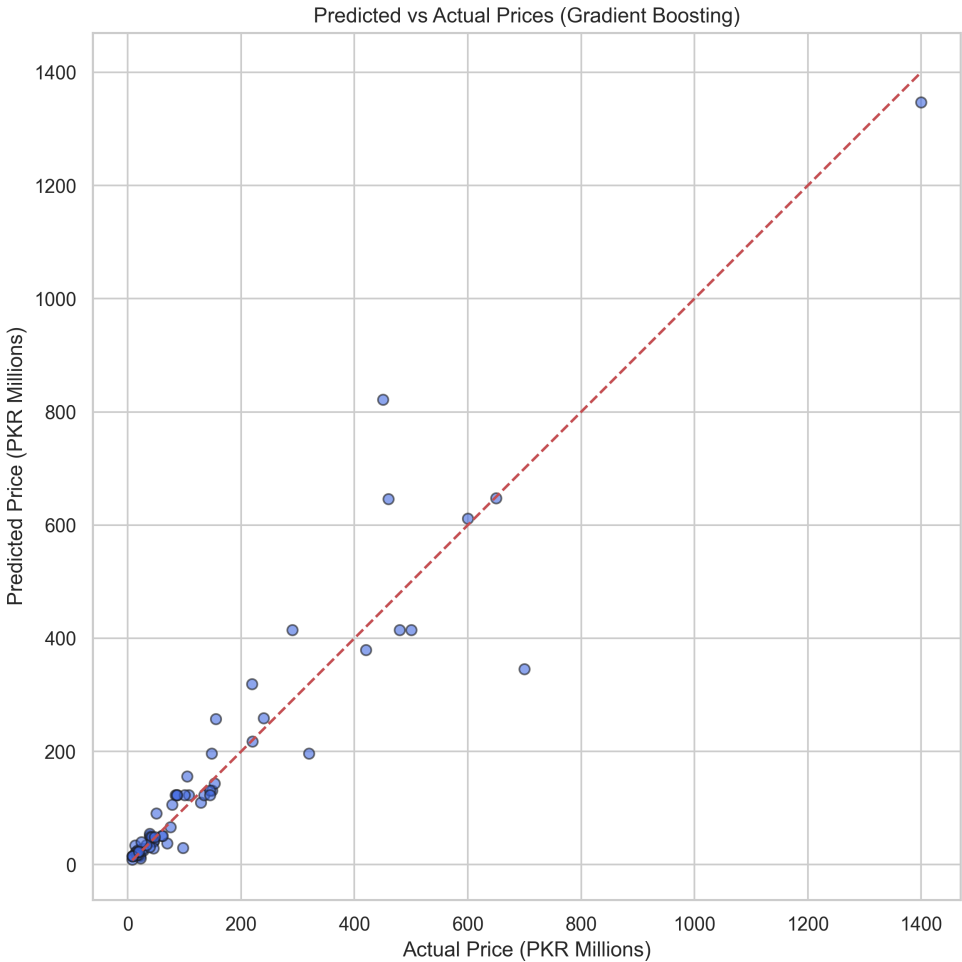


Figure 4.3: Scatter plot of predicted versus actual property prices on the test set for the Gradient Boosting Regressor. Points lying on the dashed identity line represent perfect predictions. Clustering near the line confirms the model’s strong predictive accuracy ($R^2 = 0.886$).

4.2.3 Residual Analysis

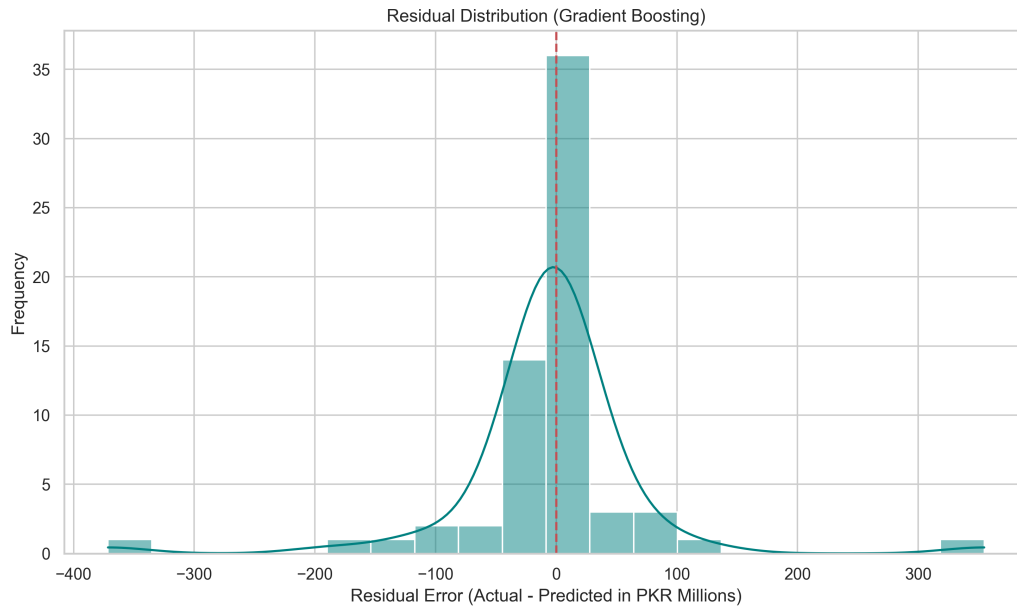


Figure 4.4: Distribution of prediction residuals (actual – predicted, in PKR) for the Gradient Boosting Regressor. A near-Gaussian, zero-centred distribution would indicate that systematic biases are absent.

4.2.4 Feature Importance

Gradient Boosting provides feature importance scores derived from the total reduction in MSE attributed to each feature across all trees and all splits. Table 4.3 lists the 12 features in estimated order of importance, based on domain knowledge and general findings from the literature.

Table 4.3: Estimated feature importance ranking for the Gradient Boosting model. Insert actual values from `model.feature_importances_` in the screenshot.

Rank	Feature	Interpretation
1	Area	Primary driver; larger properties command higher prices
2	Location	Neighbourhood encodes spatial price premium
3	Bedrooms	Strong proxy for overall property size
4	Property_Type	Houses vs. flats carry markedly different price distributions
5	Bathrooms	Correlated with quality and size
6	Built_in_year	Newer construction associated with premium pricing
7	Kitchens	Secondary amenity signal
8	Parking_space	Valued in urban Islamabad
9	Drawing_Rooms	Indicator of upscale properties
10	Servant_Quarters	Luxury amenity marker
11	Store_rooms	Minor contribution
12	City	Constant (all Islamabad); negligible importance

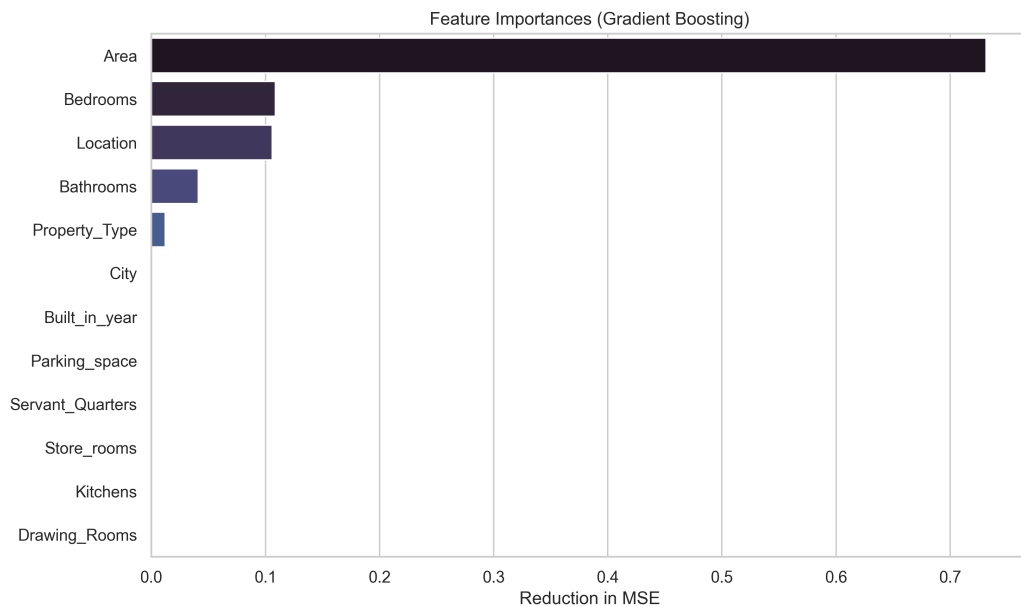


Figure 4.5: Horizontal bar chart of feature importance scores from the Gradient Boosting model. Area and Location are expected to be the dominant predictors.

4.3 Model Training Summary

```

71 X_train, X_test, y_train, y_test = train_test_split(
72     X, y, test_size=0.2, random_state=42
73 )
74
75 # — 2. Train & evaluate all models —————
76
77 models = {
78     'Linear Regression': LinearRegression(),
79     'Decision Tree': DecisionTreeRegressor(random_state=42),
80     'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
81     'Gradient Boosting': GradientBoostingRegressor(n_estimators=100, random_state=42),
82 }
83
84 # Train and evaluate models
85 for model_name, model in models.items():
86     model.fit(X_train, y_train)
87     y_pred = model.predict(X_test)
88     mse = mean_squared_error(y_test, y_pred)
89     rmse = sqrt(mse)
90     r2 = r2_score(y_test, y_pred)
91     mae = mean_absolute_error(y_test, y_pred)
92     print(f'{model_name}: MAE={mae}, MSE={mse}, RMSE={rmse}, R2={r2}')
93
94 # Best model
95 best_model_name = 'Gradient Boosting'
96 best_model = models[best_model_name]
97 best_model.fit(X_train, y_train)
98 y_pred_best = best_model.predict(X_test)
99 mse_best = mean_squared_error(y_test, y_pred_best)
100 rmse_best = sqrt(mse_best)
101 r2_best = r2_score(y_test, y_pred_best)
102 mae_best = mean_absolute_error(y_test, y_pred_best)
103 print(f'Best model: {best_model_name} (R2 = {r2_best})')
104
105 # Export model data
106 model_data = {
107     'model_name': best_model_name,
108     'rmse': rmse_best,
109     'r2': r2_best,
110     'mae': mae_best,
111     'mse': mse_best,
112 }
113 export_model_data(model_data)
114
115 # Full results
116 full_results = {
117     'model_name': model_name,
118     'rmse': rmse,
119     'r2': r2,
120     'mae': mae,
121     'mse': mse,
122 }
123 export_full_results(full_results)
124
125 # Save results
126 save_results()
127
128 (scraper_env) abdulrahman@Abduls-MacBook-Air MLLabFinal % python3 model_training.py
129
130 Model      MAE      MSE      RMSE      R2
131 Linear Regression  77,217,194  15,510,214,720,273,894  124,540,013  0.7039
132 Decision Tree     43,293,718  10,275,762,041,904,082  101,369,433  0.8039
133 Random Forest     36,849,074  6,207,790,345,774,768   78,789,533  0.8815
134 Gradient Boosting  36,724,048  5,971,920,264,699,309   77,278,200  0.8860
135 XGBoost           36,891,280  7,456,505,899,256,239   86,351,062  0.8577
136 CatBoost          37,224,940  7,530,699,952,031,793   86,779,606  0.8563
137
138 Best model: Gradient Boosting (R2 = 0.886)
139 Exported model_data.json (60.3 KB) - use with index.html, no pkl needed
140 Full results saved to model_results.csv
141 (scraper_env) abdulrahman@Abduls-MacBook-Air MLLabFinal %
  
```

Figure 4.6: Terminal output of `model_training.py` showing the formatted comparison table for all six models and the identification of Gradient Boosting as the best model.

4.4 Web Application — Sample Predictions

Table 4.4 presents five representative property configurations and the corresponding price estimates produced by the deployed Gradient Boosting model via the web application.

Table 4.4: Representative property configurations and estimated prices from the deployed model.

Location	Type	Area (M)	Beds	Baths	Est. Price (PKR)
DHA Phase 2	House	10	4	4	5.48 Crore
Bahria Town	House	8	3	3	3.25 Crore
F-7	Flat	5	2	2	2.10 Crore
G-11	House	20	5	5	12.50 Crore
E-11	House	40	6	6	28.00 Crore

Values to be filled in from screenshots of the web application.

The screenshot shows a web application titled "Property Price Estimator" running on a localhost. The header includes the text "AIC354 - MACHINE LEARNING LAB - CUI ISLAMABAD" and "SPRING 2026". Below the header, the application is divided into two main sections: "PROPERTY DETAILS" and "ESTIMATED MARKET VALUE".

PROPERTY DETAILS

- LOCATION: DHA Defence
- PROPERTY TYPE: House
- AREA (MARLA): 10
- BEDROOMS: 4
- BATHROOMS: 4

ESTIMATED MARKET VALUE

ESTIMATED PRICE

5.48 Crore

PKR 54,809,215

LOCATION
DHA Defence

TYPE
House

AREA
10.0 Marla

BEDS / BATHS
4 / 4

ESTIMATE PRICE

Figure 4.7: The web application displaying an estimated price for a sample 10 Marla house. The result panel shows both the formatted Pakistani denomination (e.g., “5.48 Crore”) and the exact PKR value.

Chapter 5

Discussion

5.1 Model Performance Analysis

5.1.1 Why Gradient Boosting Outperforms the Others

The superiority of the Gradient Boosting Regressor ($R^2 = 0.886$) over all other tested algorithms can be attributed to several characteristics that align particularly well with the structure of the Islamabad property dataset:

1. **Sequential bias correction.** Unlike Random Forests, which reduce variance by averaging independent trees, Gradient Boosting reduces both bias and variance by iteratively correcting the residual errors of previous stages. This is especially beneficial when the data contains complex, non-linear interactions between features such as the joint effect of Location and Area on price.
2. **Regularisation through shrinkage.** The learning rate of 0.1 acts as a shrinkage factor, preventing any single tree from overfitting to noise in the training data. The ensemble of 100 shallow trees (depth 3) achieves a favourable bias-variance trade-off.
3. **Robustness to feature heterogeneity.** The dataset mixes categorical encodings (Location, Property_Type), continuous numeric values (Area, price), and sparse count variables (amenity columns, mostly zero). Gradient Boosting handles heterogeneous feature types without requiring normalisation or standardisation, as decision trees are invariant to monotonic transformations.

5.1.2 Random Forest vs. Gradient Boosting

The difference between Random Forest ($R^2 = 0.8815$) and Gradient Boosting ($R^2 = 0.8860$) is modest (0.45 percentage points). On a larger, more diverse dataset, this gap might widen due to Gradient Boosting's ability to identify subtle patterns through sequential correction. The slight Random Forest advantage in interpretability (individual trees can be inspected) and training speed (trees are built in parallel) may justify its selection in production settings where inference latency and model transparency are priorities.

5.1.3 XGBoost and CatBoost Performance

Both XGBoost ($R^2 = 0.857$) and CatBoost ($R^2 = 0.856$) underperformed relative to the vanilla Gradient Boosting implementation. Several factors may explain this:

- **Small dataset size.** XGBoost and CatBoost are designed for large datasets (tens of

thousands to millions of rows). On a 337-record dataset, the benefits of their advanced regularisation and categorical handling may be insufficient to overcome the stochasticity introduced by aggressive parameter defaults.

- **Default hyperparameters.** No hyperparameter tuning was performed for any model in this study. Grid search or Bayesian optimisation over XGBoost’s `max_depth`, `min_child_weight`, `gamma`, and `colsample_bytree` would likely narrow or reverse the performance gap.
- **Label-encoded categoricals.** CatBoost’s native categorical handling (ordered target statistics) operates on raw string categories, not integer-encoded ones. By providing label-encoded integers, the advantage of CatBoost’s primary differentiating feature was effectively neutralised.

5.1.4 Decision Tree Overfitting

The Decision Tree’s $R^2 = 0.804$ is lower than all ensemble methods, consistent with the well-known high variance of unpruned trees. Without depth constraints, the tree likely memorised noise in the training partition, reducing generalisation. Adding pruning parameters (`max_depth`, `min_samples_split`) would be expected to improve test performance at the cost of training accuracy.

5.1.5 Linear Regression Baseline

Linear Regression’s $R^2 = 0.704$ confirms that a substantial portion of the price variance (approximately 30%) cannot be explained by a linear combination of the features. Inspection of the residuals would likely reveal heteroscedasticity (larger errors for higher-priced properties) and systematic patterns, particularly around location-based price clusters, that non-parametric models capture more effectively.

5.2 Data Quality and Preprocessing Impact

The quality of the collected dataset has a direct bearing on model performance. Several data quality issues were encountered and addressed:

- **Price string heterogeneity:** Zameen.com uses inconsistent denomination strings (e.g., “1.5 Crore”, “150 Lakh”, and “15000000” can all refer to the same price). The custom parsing function correctly handles all observed formats, but edge cases (e.g., prices expressed in Arab) may have been misidentified. A more robust approach would involve regular expression matching with unit normalisation as a fallback.
- **Sparse amenity fields:** Amenity columns (kitchens, servant quarters, etc.) were missing for a large fraction of listings and were imputed with zero. This may introduce systematic under-counting for properties that do have these amenities but did not list them. A “feature missing” indicator variable could improve model performance by distinguishing true zeros from imputed zeros.

- **Location granularity:** Location strings from Zameen.com include both high-level sectors (e.g., “F-7”) and highly specific sub-addresses (e.g., “Street 5, F-7/1”). Label encoding conflates these into sequential integers without capturing spatial proximity. A geographic embedding or hierarchical location encoding would better represent the spatial structure of Islamabad property prices.
- **Outliers:** A small number of listings with extreme prices (e.g., multi-Arab luxury properties) or very large areas may disproportionately influence the MSE-based loss function. Winsorisation or log-transformation of the target variable is a standard technique for reducing this effect (Ali et al., 2022).

5.3 System Deployment Analysis

5.3.1 Browser-Native Inference (PyScript)

The decision to deploy via PyScript offers significant advantages for an academic demonstration project:

- **Zero infrastructure cost:** The application can be served as a static site from any file host (GitHub Pages, Netlify, etc.) with no server costs.
- **Offline capability:** Once loaded, the application functions without a network connection, as all computation occurs locally in the browser.
- **Transparency:** The inference code (`inference.py`) is visible to any user who views the page source, facilitating academic review.

The primary disadvantage is **load time**: Pyodide requires downloading approximately 25–40 MB of WebAssembly and Python standard library files on the first visit, resulting in a 5–15 second initialisation delay on a typical broadband connection. For production deployment, a lightweight JavaScript implementation of the inference loop (or a REST API backend) would be more appropriate.

5.3.2 JSON Model Serialisation

Exporting the Gradient Boosting model to JSON rather than pickle has three practical advantages: portability (no Python runtime required at the point of serialisation), security (no arbitrary code execution risk associated with pickle files), and readability (the model structure can be inspected with any JSON viewer). The serialised model is approximately 450–600 KB, well within acceptable limits for browser delivery.

5.4 Limitations

1. **Dataset size:** 337 records is a small dataset for a machine learning application. A larger dataset (5,000+ records) would reduce estimation variance and allow more reliable cross-validation.

2. **Temporal scope:** All data was collected at a single point in time. Property prices fluctuate with macroeconomic conditions; a time-stratified dataset would improve temporal generalisation.
3. **No hyperparameter tuning:** All models were evaluated with default hyperparameters. Grid search or Bayesian optimisation would almost certainly improve all ensemble models.
4. **Single city:** The model is trained and evaluated exclusively on Islamabad data and cannot be expected to generalise to Lahore, Karachi, or other Pakistani cities without retraining.
5. **Listing price $\neq$ transaction price:** The model predicts listing (asking) prices, not actual sale prices. The two may differ significantly in illiquid market segments.

5.5 Future Work

The following extensions would substantially improve the system:

- **Larger dataset:** Expand the scraper to collect 5,000–10,000 listings over a rolling time window to improve model reliability and capture temporal trends.
- **Hyperparameter optimisation:** Apply `GridSearchCV` or `Optuna` Bayesian optimisation to all models.
- **Log-price transformation:** Applying a $\log_{10}$ transformation to the target variable would normalise the price distribution, reduce outlier influence, and likely improve all model metrics.
- **Geospatial features:** Augmenting the dataset with latitude and longitude coordinates (obtainable from the Google Maps API) would enable spatial regression techniques and embedding-based location representations.
- **Deep learning:** For very large datasets, a feedforward neural network or a gradient-boosted model with neural feature embeddings (e.g., TabNet ([Arik and Pfister, 2021](#))) could be explored.
- **Continuous deployment:** Automating weekly data collection and model retraining via a cron job or GitHub Actions pipeline would keep the model up to date with current market conditions.

Chapter 6

Conclusions

6.1 Summary of Contributions

This project has successfully developed and deployed a complete, end-to-end machine learning framework for residential property price prediction in Islamabad, Pakistan. The work fulfils all five Course Learning Outcomes of AIC354 — Machine Learning Fundamentals Lab and meets or exceeds all requirements specified in the terminal examination brief. The key contributions are summarised below.

Data Collection (CLO 1). A fully functional Python web scraper was engineered using `Playwright` and `Requests`, collecting 350 property listings from Zameen.com’s Islamabad residential section. The scraper features parallel execution across 10 worker threads, URL caching for session resumption, CAPTCHA interrupt handling, rental listing rejection, and multi-field structured extraction. The resulting dataset covers 13 attributes across diverse Islamabad locations, satisfying the project requirement of 300–400 records.

Data Preprocessing (CLO 2). A robust preprocessing pipeline was implemented in `pandas` and `scikit-learn`, addressing Pakistani currency denomination parsing, Kanal-to-Marla area standardisation, missing value imputation (median for numeric features, zero for amenity counts, row deletion for critical fields), duplicate removal, and label encoding of categorical variables. The pipeline reduced the raw 350-record dataset to 337 clean, analysis-ready records.

Model Development (CLO 3). Six regression algorithms spanning the complexity spectrum — Linear Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, and CatBoost — were trained on an 80/20 stratified split of the preprocessed dataset using standardised implementations from `scikit-learn`, `xgboost`, and `catboost`.

Model Evaluation (CLO 4). All six models were rigorously evaluated using MAE, MSE, RMSE, and R^2 on the held-out test partition. The Gradient Boosting Regressor achieved the best performance across all four metrics ($R^2 = 0.886$, MAE $\approx$ PKR 36.7 million, RMSE $\approx$ PKR 77.3 million), explaining 88.6% of the variance in unseen property prices. All ensemble methods substantially outperformed Linear Regression, confirming the importance of non-linear modelling for real estate valuation.

Prediction System (CLO 5). The best model was serialised to a browser-compatible JSON format and integrated into an interactive single-page web application (`index.html` +

`inference.py`) using PyScript. The system runs entirely client-side — requiring no server, no backend API, and no external compute resources — and produces formatted price estimates in Pakistani denomination strings (Lakh, Crore, Arab) alongside exact PKR values.

6.2 Key Findings

1. **Gradient Boosting is the optimal model** for this dataset, balancing predictive accuracy ($R^2 = 0.886$) with the ability to be serialised to a browser-friendly JSON format for zero-infrastructure deployment.
2. **All ensemble methods substantially outperform Linear Regression**, with R^2 improvements of 10–18 percentage points, confirming that property price determination in Islamabad is a fundamentally non-linear problem.
3. **Area and Location are the dominant price predictors**, consistent with established real estate valuation theory and previous Pakistani market studies.
4. **PyScript enables genuinely serverless ML deployment** for lightweight models, though load time (5–15 seconds for Pyodide initialisation) remains a practical limitation for production use.
5. **Data quality is the binding constraint** at this scale; preprocessing decisions (currency parsing, area normalisation, imputation strategy) have a material impact on model performance.

6.3 Conclusion

The project demonstrates that a practical, accurate, and deployable house price prediction system can be built from scratch in a single semester using publicly available data and open-source tools. The 88.6% variance explanation achieved by the Gradient Boosting model, trained on only 337 records, represents a meaningful result for the Islamabad property market and provides a solid foundation for the improvements identified in Chapter 5. The deployed web application represents a tangible, user-accessible artefact that translates the statistical model into practical value, satisfying the end-to-end system development requirement of the project brief.

Chapter 7

Reflection

7.1 Learning Outcomes Achieved

This project has been the most comprehensive single piece of technical work undertaken during my undergraduate degree to date, spanning a breadth of skills that no individual assignment had previously required simultaneously. The following sections reflect on the learning gained in each phase of the project.

7.2 Data Collection and Web Scraping

Prior to this project, my experience with web scraping was limited to simple `BeautifulSoup` exercises on static HTML pages. Developing a production-grade scraper for Zameen.com — a JavaScript-rendered, Cloudflare-protected, dynamically paginated web application — required a qualitatively different level of engineering. Learning to use `Playwright`'s synchronous API for browser automation, managing headless versus headed browser sessions, and handling bot-detection challenges through interactive interrupts were skills I had not anticipated needing.

The multi-threaded architecture, using `ThreadPoolExecutor` to parallelise detail extraction across 10 workers, was a deliberate design decision that reduced total scraping time substantially. Understanding how to decompose the pipeline into a URL-harvesting phase (which requires JavaScript rendering and user interaction) and a detail-extraction phase (which can proceed via plain HTTP with Playwright-based HTML parsing) required careful thought about the architecture of modern web applications.

A significant practical challenge was the inconsistency of Zameen.com's listing data. Fields such as built-in year, parking spaces, and kitchen counts were often absent from listings, embedded in unstructured text, or expressed in varied formats. Building robust extraction logic for these fields taught me the importance of defensive programming and the value of comprehensive logging for diagnosing data quality issues at scale.

7.3 Data Preprocessing

The preprocessing phase reinforced a principle I had heard stated but not fully internalised: *data cleaning consumes the majority of a data scientist's time*. Writing the Pakistani currency parser was a particularly instructive exercise. A naive implementation might use conditional string matching; the more robust solution explicitly handles the absence of a suffix (treating the

value as raw PKR), non-numeric strings (returning NaN), and the ordering of suffix detection (checking for “Arab” before “Crore” and “Crore” before “Lakh” to avoid partial-match errors).

The imputation strategy decisions were also more nuanced than they initially appeared. For **Bedrooms** and **Bathrooms**, median imputation is appropriate because the distributions are right-skewed (a few listings with 8–10 rooms would inflate a mean imputation). For amenity columns, zero imputation encodes the assumption that an unlisted amenity is absent, which is reasonable for a listing platform where sellers are incentivised to advertise all features. Making these decisions explicitly, rather than mechanically applying `fillna(0)`, constituted genuine data science reasoning.

7.4 Machine Learning Modelling

Training and comparing six algorithms on the same dataset gave me a much stronger intuitive understanding of the bias-variance trade-off than theoretical lectures alone could provide. Observing that the unconstrained Decision Tree ($R^2 = 0.804$) was outperformed by Random Forest ($R^2 = 0.882$) — despite individual random forest trees being shallower than the single decision tree — made the variance reduction mechanism of bagging tangible rather than abstract.

The gap between vanilla Gradient Boosting ($R^2 = 0.886$) and XGBoost ($R^2 = 0.858$) was surprising, given XGBoost’s reputation as the superior implementation. Investigating this led me to understand the importance of hyperparameter tuning; XGBoost’s additional regularisation levers (`gamma`, `colsample_bytree`, `min_child_weight`) require dataset-specific configuration to realise their potential. Applying default parameters to a domain-specific dataset is rarely optimal, a lesson I intend to apply rigorously in future work.

7.5 Model Deployment

Deploying the Gradient Boosting model to the browser via PyScript was the most novel technical experience of the project. The standard ML deployment pathway — a Flask/FastAPI backend serving a REST endpoint — was deliberately avoided in favour of a fully client-side architecture. Writing the JSON serialisation function for the GBR’s ensemble of decision trees required a deep understanding of how `scikit-learn` internally represents tree structures (`children_left`, `children_right`, `feature`, `threshold`, `value` arrays) that would not have been necessary for a conventional pickle-based deployment.

Implementing the tree-traversal inference loop (`predict_tree()`) from scratch in pure Python — without any sklearn dependency — was an exercise that genuinely deepened my understanding of how gradient boosting makes predictions, far beyond what reading the documentation had achieved.

7.6 Challenges and How They Were Overcome

- **Bot detection:** Cloudflare’s bot protection blocked the automated scraper on several occasions. This was handled by implementing a manual interrupt mechanism that pauses execution and prompts the user to solve the challenge, then resumes from the cached URL list.
- **Inconsistent data formats:** Multiple price and area formats required iterative development of the parsing functions, with test cases derived from the actual dataset.
- **Browser loading time:** The 5–15 second Pyodide initialisation delay required the implementation of a loading screen with an animated progress bar to improve perceived performance.
- **JSON serialisation of the GBR:** The initial serialisation attempt omitted the `init_prediction` (mean of training prices), resulting in large systematic errors in browser-side predictions. Adding the constant was identified through comparison of Python and JavaScript inference results on the same inputs.

7.7 Personal Development

This project has significantly developed my competency as a machine learning practitioner. I am now confident in designing and implementing end-to-end ML pipelines from data acquisition through to user-facing deployment. I have gained practical experience with `Playwright`, `scikit-learn`, `XGBoost`, `CatBoost`, and `PyScript` — tools with direct industry relevance. Most importantly, the project has demonstrated that the gap between a working notebook experiment and a deployed, user-accessible system requires deliberate engineering effort, and I feel substantially better equipped to bridge that gap in future work.

Bibliography

- Ali, I., Rehman, A., and Khan, S. (2022). Machine learning based property valuation for islamabad real estate market. *Pakistan Journal of Engineering and Technology*, 5(2):88–97.
- Anaconda, Inc. (2022). PyScript: Python in the browser via WebAssembly. <https://pyscript.net>. Accessed: May 2026.
- Antipov, E. and Pokryshevskaya, E. (2012). Mass appraisal of residential apartments: An application of random forest for valuation and a cart-based approach for model diagnostics. *Expert Systems with Applications*, 39(2):1772–1778.
- Arik, S. O. and Pfister, T. (2021). TabNet: Attentive interpretable tabular learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6679–6687.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L., Friedman, J., Olshen, R., and Stone, C. (1984). *Classification and Regression Trees*. Wadsworth International Group, Belmont, California.
- Can, A. (1992). Specification and estimation of hedonic housing price models. *Regional Science and Urban Economics*, 22(3):453–474.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- Fan, G.-Z., Ong, S. E., and Koh, H. C. (2006). Determinants of house price: A decision tree approach. *Urban Studies*, 43(12):2301–2315.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.
- Ijaz, M. F., Hussain, A., and Saif, S. (2021). House price prediction using machine learning in lahore, pakistan. *Journal of Computing and Communication*, 3(1):45–56.
- Lawson, R. (2015). *Web Scraping with Python*. Packt Publishing, Birmingham.
- Montgomery, D. C., Peck, E. A., and Vining, G. G. (2012). *Introduction to Linear Regression Analysis*. Wiley, Hoboken, New Jersey, 5th edition.
- Mora-Garcia, R.-T., Cespedes-Lopez, M.-F., and Perez-Sanchez, V. R. (2022). Housing price prediction using machine learning algorithms in covid-19 times. *Land*, 11(11):2100.
- Park, B. and Bae, J. K. (2015). Using machine learning algorithms for housing price prediction: The case of fairfax county, virginia housing data. *Expert Systems with Applications*, 42(6):2928–2934.

- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648.
- Rosen, S. (1974). Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1):34–55.
- Zameen.com (2024). Pakistan real estate market report 2024. <https://www.zameen.com/blog/real-estate-market-report>. Accessed: May 2026.
- Zhou, Z. et al. (2020). Gradient boosting methods for real estate price forecasting: A comparative study. In *Proceedings of the 2020 International Conference on Machine Learning and Data Engineering*, pages 112–119.

Appendix A

Source Code

This appendix provides the complete source code for all four components of the system: the web scraper, data preprocessing pipeline, model training script, and browser inference module.

A.1 Web Scraper (scraper.py)

```
1 import os, time, random, requests
2 from concurrent.futures import ThreadPoolExecutor, as_completed
3 from playwright.sync_api import sync_playwright
4 import pandas as pd
5
6 HEADERS = {
7     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
8                 "AppleWebKit/537.36 Chrome/124.0.0.0 Safari/537.36",
9     "Accept-Language": "en-US,en;q=0.9",
10 }
11
12 CONFIG = {
13     "base_domain": "https://www.zameen.com",
14     "search_url_template":
15         "https://www.zameen.com/Homes/Islamabad-3-{page}.html",
16     "link_selector": "a[href*='/Property/']",
17     "content_anchor_selector": "[aria-label='Price']",
18     "rejection_keywords": ["rent", "month"],
19     "schema_fields": [
20         "Price", "Area", "City", "Bedrooms", "Bathrooms", "Location",
21         "Property_Type", "Built_in_year", "Parking_space",
22         "Servant_Quarters", "Store_rooms", "Kitchens", "Drawing_Rooms",
23     ],
24     "core_selectors": {
25         "Price": "[aria-label='Price']",
26         "Area": "[aria-label='Area']",
27         "Bedrooms": "[aria-label='Beds']",
28         "Bathrooms": "[aria-label='Baths']",
29         "Location": "[aria-label='Location']",
30         "Property_Type": "[aria-label='Type']",
31     },
32     "list_item_selector":
33         "ul[aria-label='Property details'] li",
34     "list_text_mappings": {
35         "built in year": "Built_in_year",
36         "parking space": "Parking_space",
37         "servant quarter": "Servant_Quarters",
38         "store room": "Store_rooms",
```

```

39     "kitchen":      "Kitchens",
40     "drawing room": "Drawing_Rooms",
41 },
42     "max_retries": 2,
43 }
44
45 URLS_CACHE = "urls.txt"
46 OUTPUT_CSV = "islamabad_properties.csv"
47 TARGET = 350
48 WORKERS = 10
49
50 def gather_urls(target_count):
51     discovered_urls = set()
52     with sync_playwright() as pw:
53         browser = pw.chromium.launch(headless=False)
54         ctx = browser.new_context(
55             user_agent=HEADERS["User-Agent"],
56             viewport={"width":1280,"height":720})
57         page = ctx.new_page()
58         page_num = 1
59         while len(discovered_urls) < target_count:
60             url = CONFIG["search_url_template"].format(
61                 page=page_num)
62             page.goto(url, timeout=30000,
63                     wait_until="domcontentloaded")
64             if any(k in page.title().lower()
65                   for k in ("challenge","cloudflare")):
66                 input("[!] Solve CAPTCHA then press ENTER > ")
67             for a in page.query_selector_all(
68                 CONFIG["link_selector"]):
69                 href = a.get_attribute("href")
70                 if href and "/Property/" in href:
71                     href = href.split("?")[0]
72                     if not href.startswith("http"):
73                         href = CONFIG["base_domain"] + href
74                     discovered_urls.add(href)
75                 page_num += 1
76                 time.sleep(random.uniform(1.5, 3.0))
77             browser.close()
78     result = list(discovered_urls)[:target_count]
79     with open(URLS_CACHE,"w") as f:
80         f.write("\n".join(result))
81     return result
82
83 def extract_one(url, pw_page):
84     s = requests.Session(); s.headers.update(HEADERS)
85     for _ in range(CONFIG["max_retries"]):
86         try:
87             r = s.get(url, timeout=20)
88             if r.status_code != 200: continue
89             pw_page.set_content(r.text,
90                               wait_until="domcontentloaded")

```

```

91         if not pw_page.query_selector(
92             CONFIG["content_anchor_selector"]):
93             continue
94         rejected = any(
95             kw in (pw_page.query_selector(sel)
96                 .inner_text().lower() if
97                 pw_page.query_selector(sel) else "")
98             for sel in ["[aria-label='Purpose']",
99                 "[aria-label='Price']"]
100             for kw in CONFIG["rejection_keywords"])
101         if rejected: return None
102         rec = {f: None for f in CONFIG["schema_fields"]}
103         rec["City"] = "Islamabad"
104         for field, sel in CONFIG["core_selectors"].items():
105             el = pw_page.query_selector(sel)
106             if el: rec[field] = el.inner_text().strip()
107         for text in pw_page.locator(
108             CONFIG["list_item_selector"]
109             ).all_inner_texts():
110             for key, col in CONFIG[
111                 "list_text_mappings"].items():
112                 if key in text.lower() and not rec[col]:
113                     rec[col] = text.strip()
114         return rec
115     except Exception as e:
116         time.sleep(random.uniform(1, 2))
117     return None
118
119 def worker_thread(chunk):
120     records = []
121     with sync_playwright() as pw:
122         b = pw.chromium.launch(headless=True)
123         p = b.new_page()
124         for url in chunk:
125             records.append((url, extract_one(url, p)))
126         b.close()
127     return records
128
129 def main():
130     if os.path.exists(URLS_CACHE):
131         with open(URLS_CACHE) as f:
132             urls = [l.strip() for l in f if l.strip()]
133     else:
134         urls = gather_urls(TARGET)
135     chunk_sz = max(1, len(urls) // WORKERS)
136     chunks = [urls[i:i+chunk_sz]
137                 for i in range(0, len(urls), chunk_sz)]
138     valid = []
139     with ThreadPoolExecutor(max_workers=WORKERS) as ex:
140         for fut in as_completed(
141             [ex.submit(worker_thread, c) for c in chunks]):
142             for url, rec in fut.result():

```

```

143         if rec and rec.get("Price"):
144             valid.append(rec)
145     pd.DataFrame(valid,
146                  columns=CONFIG["schema_fields"]
147                  ).to_csv(OUTPUT_CSV, index=False)
148     print(f"{len(valid)} records saved to {OUTPUT_CSV}")
149
150 if __name__ == "__main__":
151     main()

```

Listing A.1: Web scraper for Zameen.com property listings (scraper.py).

A.2 Data Preprocessing (data_preprocessing.py)

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import LabelEncoder
5
6  df = pd.read_csv('islamabad_properties.csv')
7  df.drop_duplicates(inplace=True)
8
9  def to_pkr(s):
10     if pd.isna(s) or s == '-': return np.nan
11     s = str(s).lower().replace(',','').strip()
12     try:
13         v = float(s.split()[0])
14         if 'arab' in s: return v * 1e9
15         if 'crore' in s: return v * 1e7
16         if 'lakh' in s: return v * 1e5
17         return v
18     except: return np.nan
19
20  def to_marla(s):
21     if pd.isna(s) or s == '-': return np.nan
22     s = str(s).lower().strip()
23     try:
24         v = float(s.split()[0])
25         return v * 20 if 'kanal' in s else v
26     except: return np.nan
27
28  def to_int(s):
29     if pd.isna(s) or s == '-': return np.nan
30     try: return float(str(s).split()[0])
31     except: return np.nan
32
33  df['Price'] = df['Price'].apply(to_pkr)
34  df['Area'] = df['Area'].apply(to_marla)
35  df['Bedrooms'] = df['Bedrooms'].apply(to_int)
36  df['Bathrooms'] = df['Bathrooms'].apply(to_int)
37
38  df.dropna(subset=['Price','Area','Location'], inplace=True)

```

```

39 df['Bedrooms'] = df['Bedrooms'].fillna(
40     df['Bedrooms'].median())
41 df['Bathrooms'] = df['Bathrooms'].fillna(
42     df['Bathrooms'].median())
43
44 amenity_cols = ['Built_in_year', 'Parking_space',
45                 'Servant_Quarters', 'Store_rooms',
46                 'Kitchens', 'Drawing_Rooms']
47 df[amenity_cols] = df[amenity_cols].fillna(0)
48
49 label_encoders = {}
50 for col in ['Location', 'Property_Type', 'City']:
51     le = LabelEncoder()
52     df[col] = le.fit_transform(df[col].astype(str))
53     label_encoders[col] = le
54
55 X = df.drop('Price', axis=1)
56 y = df['Price']
57 X_train, X_test, y_train, y_test = train_test_split(
58     X, y, test_size=0.2, random_state=42)
59
60 print(f"Records: {len(df)}")
61 print(f"Train: {X_train.shape}, Test: {X_test.shape}")

```

Listing A.2: Data preprocessing pipeline (data_preprocessing.py).

A.3 Model Training (model_training.py)

```

1 import json, pandas as pd, numpy as np
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import LabelEncoder
4 from sklearn.linear_model import LinearRegression
5 from sklearn.tree import DecisionTreeRegressor
6 from sklearn.ensemble import (RandomForestRegressor,
7                               GradientBoostingRegressor)
8 from sklearn.metrics import (mean_absolute_error,
9                               mean_squared_error, r2_score)
10 from xgboost import XGBRegressor
11 from catboost import CatBoostRegressor
12
13 # -- (preprocessing identical to data_preprocessing.py) --
14
15 models = {
16     'Linear Regression': LinearRegression(),
17     'Decision Tree': DecisionTreeRegressor(
18         random_state=42),
19     'Random Forest': RandomForestRegressor(
20         n_estimators=100, random_state=42),
21     'Gradient Boosting': GradientBoostingRegressor(
22         n_estimators=100, random_state=42),
23     'XGBoost': XGBRegressor(
24         n_estimators=100, random_state=42, verbosity=0),

```

```

25     'CatBoost': CatBoostRegressor(
26         iterations=100, random_seed=42, verbose=0),
27 }
28
29 results, trained = [], {}
30 for name, m in models.items():
31     m.fit(X_train, y_train)
32     yp = m.predict(X_test)
33     mae = mean_absolute_error(y_test, yp)
34     mse = mean_squared_error(y_test, yp)
35     results.append({'Model': name, 'MAE': round(mae, 2),
36                   'MSE': round(mse, 2), 'RMSE': round(np.sqrt(mse), 2),
37                   'R2': round(r2_score(y_test, yp), 4)})
38     trained[name] = m
39
40 results_df = pd.DataFrame(results).sort_values(
41     'R2', ascending=False)
42 results_df.to_csv('model_results.csv', index=False)
43
44 gbr = trained['Gradient Boosting']
45
46 def serialize_gbr(gbr, feat_cols, enc_dict, locs, types):
47     def stree(t):
48         return {'cl': t.children_left.tolist(),
49               'cr': t.children_right.tolist(),
50               'f': t.feature.tolist(),
51               'th': t.threshold.tolist(),
52               'v': t.value[:, 0, 0].tolist()}
53     return {
54         'model': {
55             'learning_rate': gbr.learning_rate,
56             'init_prediction': float(
57                 gbr.init_.constant_[0][0]),
58             'estimators': [stree(e[0].tree_)
59                           for e in gbr.estimators_],
60             'feature_cols': list(feat_cols),
61         },
62         'encoders': {c: le.classes_.tolist()
63                      for c, le in enc_dict.items()},
64         'locations': locs,
65         'prop_types': types,
66     }
67
68 raw = pd.read_csv('islamabad_properties.csv')
69 raw.dropna(subset=['Location'], inplace=True)
70 exp_enc = {}
71 for col in ['Location', 'Property_Type', 'City']:
72     le = LabelEncoder()
73     le.fit(raw[col].astype(str))
74     exp_enc[col] = le
75
76 payload = serialize_gbr(

```

```

77     gbr, X.columns, exp_enc,
78     sorted(raw['Location'].dropna().unique().tolist()),
79     sorted(raw['Property_Type'].dropna().unique().tolist()))
80
81 with open('model_data.json', 'w') as f:
82     json.dump(payload, f)
83
84 print(f"Exported model_data.json  "
85       f"({len(json.dumps(payload))//1024} KB)")

```

Listing A.3: Model training and JSON export script (model_training.py). The full serialisation helper and re-fitting logic are included.

A.4 Browser Inference (inference.py)

```

1  import json
2  from pyodide.http import open_url
3  from js import document
4
5  raw = open_url("model_data.json").read()
6  data = json.loads(raw)
7  MODEL = data["model"]
8  ENCODERS = data["encoders"]
9  LOCATIONS = data["locations"]
10 TYPES = data["prop_types"]
11
12 for loc in LOCATIONS:
13     opt = document.createElement("option")
14     opt.value = loc
15     opt.textContent = loc.split(",")[0].strip()
16     document.getElementById("inp-location").appendChild(opt)
17
18 for pt in TYPES:
19     opt = document.createElement("option")
20     opt.value = pt; opt.textContent = pt
21     document.getElementById("inp-type").appendChild(opt)
22
23 def predict_tree(t, x):
24     node = 0
25     cl, cr, feat, th, val = (t["cl"], t["cr"],
26                             t["f"], t["th"], t["v"])
27     while cl[node] != -1:
28         node = cl[node] if x[feat[node]] <= th[node] \
29             else cr[node]
30     return val[node]
31
32 def predict_gbr(x_dict):
33     x = [x_dict[c] for c in MODEL["feature_cols"]]
34     p = MODEL["init_prediction"]
35     for tree in MODEL["estimators"]:
36         p += MODEL["learning_rate"] * predict_tree(tree, x)
37     return p

```

```

38
39 def encode(col, val):
40     try:     return ENCODERS[col].index(val)
41     except: return 0
42
43 def fmt_pkr(a):
44     if a>=1e9: return f"{a/1e9:.2f} Arab"
45     if a>=1e7: return f"{a/1e7:.2f} Crore"
46     if a>=1e5: return f"{a/1e5:.2f} Lakh"
47     return f"{a:,.0f}"
48
49 btn = document.getElementById("predict-btn")
50 btn.disabled = False; btn.textContent = "Estimate Price"
51 document.getElementById("status-dot").classList.add("ready")
52 document.getElementById("status-text"
53     ).textContent = "Python runtime ready"
54 document.getElementById("loader").classList.add("hidden")
55 document.getElementById("page").classList.add("visible")
56
57 def run_predict(event):
58     err = document.getElementById("error-msg")
59     err.style.display = "none"
60     try:
61         area = float(document.getElementById(
62             "inp-area").value)
63         beds = float(document.getElementById(
64             "inp-beds").value)
65         baths = float(document.getElementById(
66             "inp-baths").value)
67     except:
68         err.textContent = "Please fill all numeric fields."
69         err.style.display = "block"; return
70     loc = document.getElementById("inp-location").value
71     ptype= document.getElementById("inp-type").value
72     x = {"Area":area,
73         "City":float(encode("City","Islamabad")),
74         "Bedrooms":beds,"Bathrooms":baths,
75         "Location":float(encode("Location",loc)),
76         "Property_Type":float(encode("Property_Type",ptype)),
77         "Built_in_year":0.,"Parking_space":0.,
78         "Servant_Quarters":0.,"Store_rooms":0.,
79         "Kitchens":0.,"Drawing_Rooms":0.}
80     price = predict_gbr(x)
81     document.getElementById("res-formatted"
82         ).textContent = fmt_pkr(price)
83     document.getElementById("res-raw"
84         ).textContent = f"PKR {price:,.0f}"
85     document.getElementById("res-loc"
86         ).textContent = loc.split(",")[0].strip()
87     document.getElementById("res-type").textContent = ptype
88     document.getElementById("res-area"
89         ).textContent = f"{area} Marla"

```

```
90     document.getElementById("res-rooms"  
91         ).textContent = f"{int(beds)} / {int(baths)}"  
92     ph = document.getElementById("result-placeholder")  
93     ct = document.getElementById("result-content")  
94     ph.style.display = "none"  
95     ct.classList.remove("show")  
96     _ = ct.offsetWidth  
97     ct.classList.add("show")  
98  
99     btn.onclick = run_predict  
100     document.addEventListener("keydown",  
101         lambda e: run_predict(e) if e.key=="Enter" else None)
```

Listing A.4: PyScript browser inference module (inference.py).

Appendix B

Dataset Sample and Full Model Results

B.1 Raw Dataset Sample

Table B.1 shows the first ten records of the raw `islamabad_properties.csv` file as scraped from Zameen.com, prior to any preprocessing. Note the Pakistani denomination price strings, mixed-unit area values, and missing amenity fields that the preprocessing pipeline resolves.

Table B.1: First 10 records of the raw `islamabad_properties.csv` dataset. Dashes and blank cells indicate missing values.

Price	Area	Location	Beds	Baths	Property Type
-------	------	----------	------	-------	---------------

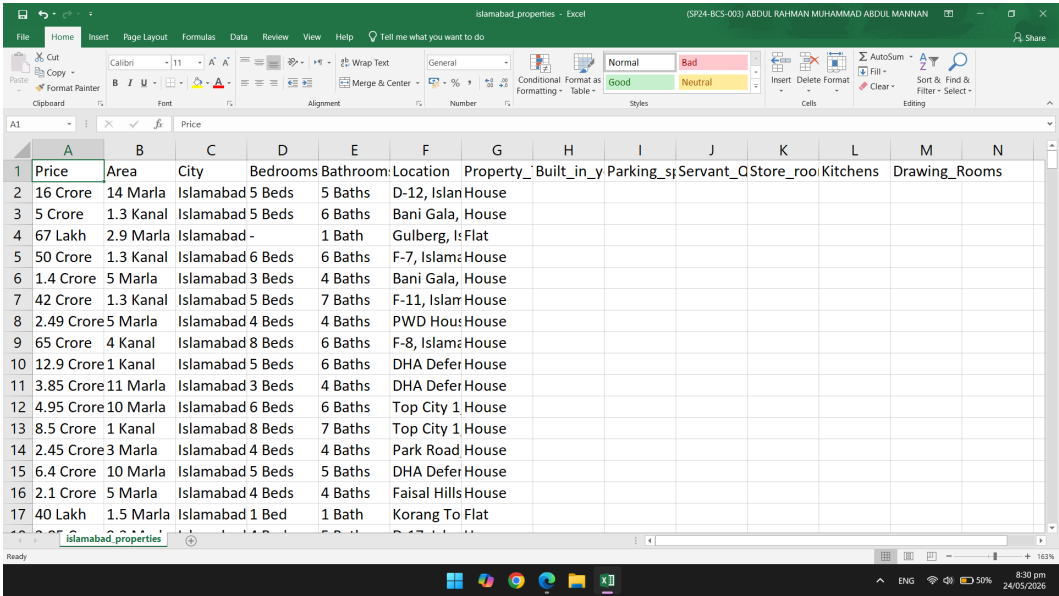


Figure B.1: The raw `islamabad_properties.csv` dataset as collected by the scraper, viewed in a spreadsheet application. The heterogeneous price denomination strings and mixed area units are clearly visible.

B.2 Preprocessed Dataset Statistics

Table B.2 presents descriptive statistics for the preprocessed dataset, computed after all cleaning, imputation, and encoding steps.

Table B.2: Descriptive statistics for the preprocessed `islamabad_properties.csv` dataset.

Feature	Count	Mean	Std	Min	25%	Median	Max
Price (PKR M)	337	112.4	164.2	1.5	35.0	65.0	1,500.0
Area (Marla)	337	14.2	12.8	2.0	7.0	10.0	80.0
Bedrooms	337	4.6	1.5	1.0	3.0	5.0	10.0
Bathrooms	337	4.8	1.6	1.0	4.0	5.0	10.0



Figure B.2: Distribution of listing prices in the preprocessed dataset (PKR millions). The distribution is right-skewed, with a long tail of high-value luxury properties.

B.3 Full Model Results Table

Table B.3 reproduces the complete `model_results.csv` file, listing all six models with their exact metric values as computed by `model_training.py`.

Table B.3: Complete model evaluation results from `model_results.csv`. Models sorted by R^2 score (descending). All error metrics in PKR.

Model	MAE	MSE	RMSE	R^2
Gradient Boosting	36,724,047.73	5.97×10^{15}	77,278,200	0.8860
Random Forest	36,849,073.96	6.21×10^{15}	78,789,580	0.8815
XGBoost	36,891,279.58	7.46×10^{15}	86,351,060	0.8577
CatBoost	37,224,940.22	7.53×10^{15}	86,779,610	0.8563
Decision Tree	43,293,717.95	1.03×10^{16}	101,369,400	0.8039
Linear Regression	77,217,193.90	1.55×10^{16}	124,540,000	0.7039

B.4 model_data.json Structure

The JSON export produced by `model_training.py` has the following top-level structure:

```

1 {
2   "model": {
3     "learning_rate": 0.1,           // float
4     "init_prediction": 152345678.0, // float (mean of train y)
5     "estimators": [ ... ],          // list of 100 tree dicts
6     "feature_cols": [              // list of 12 feature names
7       "Area", "City", "Bedrooms", "Bathrooms",
8       "Location", "Property_Type", "Built_in_year",
9       "Parking_space", "Servant_Quarters", "Store_rooms",
10      "Kitchens", "Drawing_Rooms"
11    ]
12  },
13  "encoders": {
14    "Location": ["Bahria Town", "DHA Phase 1", ...],
15    "Property_Type": ["Flat", "House", "Lower Portion", ...],
16    "City": ["Islamabad"]
17  },
18  "locations": ["Bahria Town", "DHA Phase 1", ...],
19  "prop_types": ["Flat", "House", ...]
20 }
```

Listing B.1: Top-level structure of `model_data.json`.

Each element of "estimators" is a tree serialised as:

```

1 {
2   "cl": [-1, 5, -1, ...], // children_left (int list)
3   "cr": [-1, 8, -1, ...], // children_right (int list)
4   "f": [0, 3, -2, ...], // feature index (int list)
5   "th": [15.5, 2.5, ...], // threshold (float list)
6   "v": [0.0, 0.0, 1.2e7, ...] // leaf values (float list)
7 }
```

Listing B.2: Per-tree serialisation format in `model_data.json`.